



Epic to Release — the complete simulation workflow

Every click needed to take one business requirement from intake to a released, documented delivery — fully offline, deterministic, and repeatable. The engine drives real gates, approvals and ledgers at every step; nothing is a mock-up of the governance.

0 · Before you start

- Launch the Control Centre: `demo/run_control.sh` → open `http://127.0.0.1:8720`.
- In the header, pick the **Environment: Demo or Simulation** — selecting one creates a fresh run. Both are pre-grounded with five connected SponsorConnect repositories and a routing verdict; both run the same scripted Sync storyline. The only difference is labelling: demo runs badge artifacts with a neutral **DEMO** chip, simulation runs say **SIMULATED**.
- The **acting role** selector is top-right. Gates genuinely check the role — the steps below name who must be acting for each click.
- A run's state persists on disk: reloading the page never loses your place, and a restart resumes where you left off.

Role you will switch to	What only they can do
Business Owner	Upload the requirement, answer clarifications, pass the intake gate, sign the plan (G1), approve the release
Product Analyst	Run intake analysis, create the epic
Delivery Lead	Generate the story plan, publish delivery packs, assign developers, click Sync
Engineering Lead	Generate and accept the architecture, generate delivery packs
QA Lead	Approve each pack's test plan, run quality checks, decide G3
Release Manager	Request release approval, deploy, generate the release document
Support Lead	Accept the transition to maintenance

1 · Intake — the requirement arrives

1. As **Business Owner**, on the **Intake** page use **Browse File** to upload `demo/requirement-doc/MapleSure-Disability-Claim-Submission-Requirement.pdf`. The extraction card fills with the parsed title, business objective and the seven numbered requirements — a real, deterministic parse of the document you just uploaded.
2. Scroll to **Analysis & Governance** (already open): the five **Connected Repositories** with head commits and file counts, and the **Requirement Routing** verdict — *Fits connected repos*.
3. As **Product Analyst**, run the analysis (**Regenerate Analysis**). The analysis raises its own **clarification popup** — five questions for the business (complete packet, mandatory attachments, status values, pre-population, retention). Answer as **Business Owner**, or **Answer later**.
4. As **Product Analyst**, click **Generate Epic** → **EPIC-S7-001**.

5. As **Business Owner** , click **Pass Intake Gate** .

2 · Planning — epic to stories, and Gate 1

1. As **Delivery Lead** , on **Planning** click **AI Suggest Stories** → seven stories (US-001...US-007) with acceptance criteria, owning teams, estimates, sprints and dependencies. Show the **Dependency Map** — its waves are enforced later in the build.
2. As **Business Owner** , type the approver name and click **Approve Plan & Lock** . Gate 1 signs, the plan locks, and downstream generation is now authorised — never auto-run.

3 · Build & Review — governed context, then evidence

1. As **Engineering Lead** : **Generate Architecture** , review the five-file pack, then accept it (named acceptance).
2. **Generate Delivery Packs** — one governed context pack per team, referencing plan and architecture by version.
3. As **QA Lead** , open each pack and **Approve Test Plan** — publication refuses until the AC-derived test plan is signed.
4. As **Delivery Lead** , **Publish All** — in this environment publication records a deterministic pseudo-commit; no real git is touched, and the record says so.
5. Developer workspaces provision automatically; assign the developers (Priya Raman, Marcus Chen, ...) from the workspace cards.

4 · The Sync storyline — iterative task completion

On **Developer Workspaces** (or **Build & Test Evidence**), as **Delivery Lead** , click **Sync from Git** repeatedly. Each sync completes the next task through the full lifecycle — tests, development, verification, independent review — and the toast names what completed:

Sync	What the room sees
1	US-001 (submission record) completes green; its acceptance criteria pass
2	US-002 (sponsor-scoped lookup) completes
3	US-003 arrives with a failed git push — the only red item on screen. Syncing again re-reports the same failure; the storyline will not advance past it
Rerun	Click Rerun US-003 sync — the correction cycle runs, the push succeeds, the story goes green
4	US-004 and US-005 advance together — parallel iteration across two teams
5	US-006 and US-007 complete — every acceptance criterion across the plan now passes

The US-003 beat is the governance story in miniature: the first implementation misreads "on or before" as "before", the targeted tests mirror the same misreading and stay green — and the **independent reviewer** verifies against the criterion text, not the tests, and blocks. No phase self-approves.

5 · Final gating — quality decides

1. As **QA Lead** , on **Final Gating** run the quality checks — coverage per acceptance criterion, review status, test results — then decide the gate. G3 passes only because the evidence supports it.

6 · Release — approvals, deploy, handover

1. As **Release Manager**, click **Request release approval**.
2. Record all five approvals, switching the acting role for each: **Business Owner**, **Engineering Lead**, **QA Lead**, **Release Manager**, **Support Lead** — name in the approver field, then **Approve as current role**.
3. As **Release Manager**: **Deploy to production (Release Manager)** — blue-green behind the `sponsor_claim_submission` flag.
4. As **Support Lead**: **Complete transition to maintenance** — runbook, monitoring alerts, hypercare. Gate 4 evaluates: all gates green, all approvals present, nothing stale.

7 · The Release & Design document

1. Still on **Release**, as **Release Manager**, click **Generate release document**.
2. **Open document** shows the MapleSure-themed page: table of contents, the epic and objective, plan approvals, architecture acceptance, then **per user story** — who developed it, who signed the test plan, the independent-review verdict, the changes made (including US-003's correction), and every acceptance criterion with its result — closing with the release approvals, deployment and handover.
3. **Download markdown** gives the same content as a portable file. Print the HTML page to PDF for a distributable copy.

8 · Recovery and reset

- Reloading the page never loses the storyline — its state is stored with the run.
- Syncing past the end reports *storyline complete*; it is never an error.
- A fresh start is one click: pick the environment again in the header to create a new run, or reset the current run from **Settings**.
- Everything above runs offline with no API key — hard rule 5: a beat that is adequate five times in five.

MapleSure Insurance is a fictional insurer. This walkthrough and every artifact it produces are synthetic demonstration data; no client data, PII, or client-identifiable information is involved.